

COURSE MATERIAL



Module 3: Advanced SQL Queries

Module 3 - 8 Hours

Advanced SQL Queries

Module 3 at a glance

Duration	8 hours (4 hours theory, 4 hours lab)
Prerequisites	Modules 1 and 2, with <code>fixed_deposit</code> loaded
You will build	The ICICI Bank monthly branch performance pack
You will carry forward	Join and aggregation skills used in every remaining module

Learning outcomes

By the end of this module you can:

1. Join two or more tables and explain what the `ON` clause does.
2. Choose between inner, left, right and full outer joins, and recognise the bug that turns an outer join into an inner one.
3. Use a self join to walk a hierarchy stored in a single table.
4. Aggregate with `GROUP BY`, filter groups with `HAVING`, and pivot with conditional aggregation.
5. Write scalar, list, correlated and `EXISTS` subqueries, and choose between a subquery and a join.
6. Combine result sets with `UNION`, `INTERSECT` and `EXCEPT`.

The business problem

Ticket ICICI-4530 - Monthly branch performance pack

"Every month I ask three people for numbers and get three answers. I need one report, from one source, that tells me per branch: customers, accounts and total deposits held; money in and money out this quarter by channel; customers who have not transacted at all; the ten largest transactions; and deposits maturing soon so we can call before the competition does."

1. Joins

1.1 Why joins exist

Module 2 stored a `customer_id` in `fixed_deposit` rather than the customer's name. That is deliberate: the name is stored once, in `customer`, so it can never disagree with itself. Module 4 explains the theory. The practical consequence is that **no useful banking report reads a single table**.

A join re-assembles facts that were deliberately separated.

customer	account	branch
-----	-----	-----
customer_id 1001	account_id 9001	branch_id 1
first_name Aarav	customer_id 1001	-----> branch_id 1
last_name Sharma	balance 35859.46	branch_name Fort Mumbai Main

1.2 The Cartesian product

```
SELECT c.first_name, a.account_number
FROM   customer c, account a;      -- no join condition
```

Thirty customers times thirty-six accounts is 1,080 rows: every customer paired with every account, almost all of them meaningless. This is a **Cartesian product** or **cross join**.

It is occasionally intentional, for example to generate every combination of branch and month for a report skeleton. When it is intentional, say so:

```
SELECT b.branch_name, m.month_name
FROM   branch b CROSS JOIN month_list m;
```

On production volumes an accidental cross join is serious. Thirty thousand customers against forty thousand accounts is 1.2 billion rows, and the query will consume the server until someone kills it.

1.3 INNER JOIN

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number,
       a.account_type,
       a.balance
FROM   customer c
       INNER JOIN account a ON a.customer_id = c.customer_id
ORDER BY c.customer_id;
```

The `ON` clause states how rows correspond. Only pairs satisfying it survive. `INNER` is the default, so plain `JOIN` means the same thing.

An inner join drops rows that have no match on the other side. In this query, a customer holding no account would disappear. An account with no customer cannot exist at all, because the foreign key forbids it.

1.4 Aliases

```
-- Ambiguous: customer_id exists in both tables
SELECT customer_id FROM customer JOIN account ON account.customer_id = customer.customer_id;
-- ERROR: column reference "customer_id" is ambiguous
```

Give every table a short alias and qualify every column with it. It removes the ambiguity, it shortens the query, and when someone later adds a column to one of the tables it cannot silently change which column your query means.

1.5 Joining several tables

Each additional table needs one more **ON** condition. **n tables require n-1 join conditions**. Miss one and you get a Cartesian product between that pair.

```
SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       t.txn_date, t.txn_type, t.amount, t.channel
FROM   branch      b
       JOIN customer c ON c.home_branch_id = b.branch_id
       JOIN account  a ON a.customer_id   = c.customer_id
       JOIN account_txn t ON t.account_id   = a.account_id
WHERE  t.txn_date >= DATE '2026-03-01'
       AND t.amount >= 100000
ORDER BY t.amount DESC;
```

Four tables, three conditions. Note that **account** appears only in the **FROM** clause: it is needed to connect customers to transactions, even though none of its columns is displayed. This is common and correct.

1.6 Outer joins

Type	Keeps
INNER JOIN	Only rows matching on both sides
LEFT JOIN	Every left row, plus matching right rows
RIGHT JOIN	Every right row, plus matching left rows
FULL OUTER JOIN	Every row from both sides
CROSS JOIN	Every combination, no ON clause

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number, a.balance
FROM   customer c
       LEFT JOIN account a ON a.customer_id = c.customer_id
ORDER BY c.customer_id;
```

Where a customer has no account, **account_number** and **balance** come back NULL. The word **OUTER** is optional.

RIGHT JOIN is rare in practice. Swapping the table order and using **LEFT JOIN** reads more naturally, and consistent direction makes a long query easier to review. **FULL OUTER JOIN** exists in PostgreSQL, Oracle and

SQL Server; MySQL requires a `UNION` of two outer joins to emulate it.

1.7 The bug that silently converts an outer join

```
-- Intended as an outer join, behaves as an inner join
SELECT c.customer_id, a.account_number
FROM   customer c
      LEFT JOIN account a ON a.customer_id = c.customer_id
WHERE  a.status = 'ACTIVE';

-- Correct
SELECT c.customer_id, a.account_number
FROM   customer c
      LEFT JOIN account a ON a.customer_id = c.customer_id
                        AND a.status = 'ACTIVE';
```

For an unmatched customer, every column of `a` is `NULL`, so `a.status = 'ACTIVE'` evaluates to `NULL`, and `WHERE` discards the row. The outer join did its job and the `WHERE` clause undid it.

The rule: on an outer join, conditions on the **optional** table belong in `ON`. Conditions on the **required** table belong in `WHERE`. This is one of the most common defects in reporting SQL, and it is silent - the query runs, it just quietly returns fewer rows than the business expects.

1.8 The anti-join

To find what is missing, join and then keep only the failures.

```
SELECT a.account_id, a.account_number, a.status
FROM   account a
      LEFT JOIN account_txn t ON t.account_id = a.account_id
WHERE  t.txn_id IS NULL;
```

```
account_id | account_number | status
-----+-----+-----
9001 | ICIC000009001 | DORMANT
```

Always test the joined table's **primary key** for `NULL`. Testing a nullable column would also match rows that did join but happen to hold a `NULL` there, which is a different question.

Business question	Pattern
Accounts with no transaction ever	<code>LEFT JOIN account_txn ... WHERE t.txn_id IS NULL</code>
Customers holding no deposit	<code>LEFT JOIN fixed_deposit ... WHERE fd.deposit_id IS NULL</code>
Branches with no loans this quarter	Same shape, with the date filter in <code>ON</code>

1.9 Self join

The ICICI Bank reporting hierarchy is stored in one table: `employee.manager_id` points at `employee.employee_id`.

```

SELECT e.employee_id,
       e.first_name || ' ' || e.last_name AS employee,
       e.designation,
       COALESCE(m.first_name || ' ' || m.last_name, '(no manager)') AS reports_to
FROM   employee e
       LEFT JOIN employee m ON m.employee_id = e.manager_id
WHERE  e.branch_id = 1
ORDER BY e.employee_id;

```

employee_id	employee	designation	reports_to
501	Meera Menon	Branch Manager	(no manager)
502	Sneha Nair	Relationship Manager	Meera Menon
503	Arjun Gupta	Teller	Meera Menon

Two aliases on the same table, `e` for the employee and `m` for the manager. `LEFT JOIN` is essential: a Branch Manager has a NULL `manager_id` and would vanish from an inner join.

A self join reaches exactly one level. Walking an arbitrary number of levels requires a **recursive query**, which is Module 6.

1.10 Non-equi joins

`ON` accepts any condition, not only equality.

```

SELECT t.txn_id, t.amount, b.band_name
FROM   account_txn t
       JOIN txn_band b ON t.amount >= b.min_amount
                          AND t.amount < b.max_amount;

```

Banking is full of these: interest rate slabs by balance, tax brackets, risk bands, tiered transaction charges, and card reward tiers. A small lookup table of ranges joined with a non-equi condition beats a long `CASE` expression, because the bands can be changed as data rather than as code.

2. Aggregate functions and GROUP BY

2.1 The aggregate functions

```
SELECT COUNT(*)           AS accounts,
       SUM(balance)       AS total_deposits,
       ROUND(AVG(balance), 2) AS average_balance,
       MIN(balance)       AS smallest,
       MAX(balance)       AS largest
FROM   account
WHERE  status = 'ACTIVE';
```

2.2 The three forms of COUNT

Form	Counts
COUNT(*)	Every row, regardless of NULLs
COUNT(column)	Rows where that column is not NULL
COUNT(DISTINCT column)	Distinct non-null values

Every aggregate except COUNT(*) ignores NULLs. This matters most for **AVG**, which divides by the count of non-null values, not by the number of rows.

```
SELECT AVG(interest_rate)           AS avg_of_known_rates,
       AVG(COALESCE(interest_rate, 0)) AS avg_treating_unknown_as_zero
FROM   account;
```

Neither is wrong. They answer different questions: "the average of the rates we know" versus "the average rate across the whole book". Regulatory reporting errors have been caused by not stating which was meant.

2.3 GROUP BY

```
SELECT a.branch_id,
       COUNT(*)           AS accounts,
       SUM(a.balance)     AS total_deposits,
       ROUND(AVG(a.balance),2) AS avg_balance
FROM   account a
WHERE  a.status = 'ACTIVE'
GROUP BY a.branch_id
ORDER BY total_deposits DESC;
```

```
branch_id | accounts | total_deposits | avg_balance
-----+-----+-----+-----
6 | 5 | 1302211.98 | 260442.40
4 | 4 | 1223774.69 | 305943.67
2 | 5 | 1175186.70 | 235037.34
8 | 3 | 1079395.65 | 359798.55
3 | 4 | 961233.96 | 240308.49
1 | 4 | 895617.95 | 223904.49
5 | 5 | 865546.44 | 173109.29
```

```
7 |      3 |      262050.93 |      87350.31
```

The rule: every column in the `SELECT` list must either appear inside an aggregate function or be listed in `GROUP BY`.

```
SELECT branch_id, account_number, SUM(balance) FROM account GROUP BY branch_id;
-- ERROR: column "account.account_number" must appear in the GROUP BY clause
--        or be used in an aggregate function
```

Branch 1 has several accounts. The output has one row for branch 1. Which single `account_number` should it show? The question has no answer, so the engine refuses to guess. Older MySQL versions did guess, silently, and produced a great many wrong reports.

2.4 Grouping by several columns

```
SELECT a.branch_id, a.account_type, COUNT(*) AS accounts, SUM(a.balance) AS total
FROM   account a
GROUP BY a.branch_id, a.account_type
ORDER BY a.branch_id, a.account_type;
```

One row per combination **that exists in the data**. `GROUP BY` never invents an empty combination, so a branch with no current accounts simply has no such row. If the report needs a zero row there, generate the combinations with a cross join and outer-join the data onto it.

2.5 WHERE versus HAVING

```
SELECT a.branch_id, COUNT(*) AS accounts, SUM(a.balance) AS total_deposits
FROM   account a
WHERE  a.status = 'ACTIVE'           -- filters rows, before grouping
GROUP BY a.branch_id
HAVING SUM(a.balance) > 1000000      -- filters groups, after aggregating
ORDER BY total_deposits DESC;
```

	WHERE	HAVING
Filters	Individual rows	Whole groups
Runs	Before <code>GROUP BY</code>	After <code>GROUP BY</code>
May reference aggregates	No	Yes

Put a condition in `WHERE` whenever it can go there. Fewer rows reach the grouping step, so less work is done. `HAVING` is only for conditions on the aggregated result.

2.6 Conditional aggregation

This is how you pivot rows into columns in plain SQL.

```
SELECT b.branch_name,
       SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END) AS money_in,
       SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END) AS money_out,
       COUNT(*) AS txn_count
FROM   account_txn t
```

```
        JOIN account a ON a.account_id = t.account_id
        JOIN branch b ON b.branch_id = a.branch_id
WHERE   t.status = 'SUCCESS'
GROUP  BY b.branch_name
ORDER  BY money_in DESC;
```

PostgreSQL offers a cleaner form for counts:

```
COUNT(*) FILTER (WHERE t.channel = 'UPI') AS upi_txns
```

`SUM(CASE WHEN ...)` works on every engine and is the portable choice.

2.7 Subtotals with ROLLUP

```
SELECT COALESCE(b.branch_name, 'ALL BRANCHES') AS branch,
       COALESCE(a.account_type, 'ALL TYPES')    AS account_type,
       SUM(a.balance)                          AS total_deposits
FROM   account a JOIN branch b ON b.branch_id = a.branch_id
GROUP  BY ROLLUP (b.branch_name, a.account_type)
ORDER  BY b.branch_name, a.account_type;
```

`ROLLUP` adds a subtotal row for each branch and one grand total. In a rollup row the grouped column is `NULL`, meaning "all values", which is why `COALESCE` supplies a label. Available in PostgreSQL, Oracle and SQL Server.

3. Subqueries

Kind	Returns	Used in
Scalar	One row, one column	SELECT, WHERE, SET
List	One column, many rows	IN, NOT IN, ANY, ALL
Table	Many rows and columns	FROM, as a derived table
Correlated	Re-evaluated per outer row	EXISTS, NOT EXISTS, SELECT

3.1 Scalar subqueries

```
SELECT account_number, balance
FROM   account
WHERE  balance > (SELECT AVG(balance) FROM account)
ORDER BY balance DESC;
```

Eighteen of the thirty-six accounts are above the bank-wide average of 220,372.35. If a scalar subquery ever returns more than one row, the statement fails at runtime - which is a feature, because the alternative is a silently arbitrary answer.

3.2 IN and the NOT IN trap

```
SELECT a.account_number, a.balance
FROM   account a
WHERE  a.customer_id IN (SELECT customer_id FROM customer WHERE risk_rating = 'HIGH');
```

`NOT IN` is dangerous when the subquery can return a `NULL`:

```
customer_id NOT IN (1001, 1005, NULL)

expands to
customer_id <> 1001 AND customer_id <> 1005 AND customer_id <> NULL
                                     ^ always NULL, never TRUE
```

`TRUE AND NULL` is `NULL`, and `WHERE` keeps only `TRUE`, so **the query returns no rows at all**. Not fewer rows - none. Either add `WHERE column IS NOT NULL` to the subquery or, better, use `NOT EXISTS`.

3.3 EXISTS and NOT EXISTS

```
-- Customers holding at least one fixed deposit
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
WHERE  EXISTS (SELECT 1 FROM fixed_deposit fd WHERE fd.customer_id = c.customer_id);

-- Customers holding none: ten of the thirty
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
WHERE  NOT EXISTS (SELECT 1 FROM fixed_deposit fd WHERE fd.customer_id = c.customer_id);
```

`EXISTS` asks only whether at least one row exists, so the engine can stop at the first match. The `SELECT 1` is conventional; the column list is never evaluated. `NOT EXISTS` behaves correctly with NULLs, which is why it is preferred over `NOT IN`.

3.4 Correlated subqueries

```
SELECT c.customer_id, c.first_name,
       (SELECT MAX(t.amount)
        FROM   account_txn t JOIN account a ON a.account_id = t.account_id
        WHERE  a.customer_id = c.customer_id) AS largest_txn
FROM   customer c
ORDER BY largest_txn DESC NULLS LAST;
```

The inner query references `c` from the outer query, so conceptually it runs once per customer. That is readable and perfectly fine for thirty customers. On thirty million it is a performance problem, and Module 5 shows how to rewrite it as a join. Modern optimisers often perform that rewrite themselves, but you cannot rely on it.

3.5 Derived tables

```
SELECT branch_id, accounts, total_deposits
FROM   (SELECT branch_id, COUNT(*) AS accounts, SUM(balance) AS total_deposits
        FROM   account
        WHERE  status = 'ACTIVE'
        GROUP BY branch_id) AS branch_summary
WHERE  total_deposits > 1000000
ORDER BY total_deposits DESC;
```

Aggregate first, then filter the aggregated result as though it were a table. Every derived table needs an alias. Module 6 replaces this pattern with a common table expression, which reads considerably better once queries grow.

3.6 Choosing between a subquery and a join

Situation	Prefer
You need columns from both tables	Join
You only need to test existence	<code>EXISTS</code>
You need one aggregate value to compare against	Scalar subquery
The subquery may return NULL	<code>NOT EXISTS</code> , never <code>NOT IN</code>
A join would duplicate outer rows	<code>EXISTS</code>

```
-- A join duplicates: a customer with 3 deposits appears 3 times
SELECT c.customer_id FROM customer c JOIN fixed_deposit fd ON fd.customer_id = c.customer_id;

-- EXISTS returns each customer once
SELECT c.customer_id FROM customer c
WHERE EXISTS (SELECT 1 FROM fixed_deposit fd WHERE fd.customer_id = c.customer_id);
```

4. Set operators

4.1 UNION and UNION ALL

```
SELECT customer_id, 'FIXED DEPOSIT' AS product FROM fixed_deposit
UNION ALL
SELECT customer_id, 'LOAN'          AS product FROM loan
ORDER BY customer_id;
```

	UNION	UNION ALL
Duplicates	Removed	Kept
Cost	Sorts or hashes to de-duplicate	None
Use when	Distinct rows are required	Rows cannot collide, or duplicates are meaningful

Default to **UNION ALL**. On a large result set, de-duplication is expensive, and in a report like the one above a duplicate is meaningful anyway: a customer holding two deposits genuinely has two products.

4.2 The rules

- Both queries must return the same number of columns.
- Corresponding columns must have compatible types.
- Column names come from the first query.
- **ORDER BY** appears once, at the very end, and applies to the combined result.

4.3 INTERSECT and EXCEPT

```
-- Customers holding both a deposit and a loan
SELECT customer_id FROM fixed_deposit
INTERSECT
SELECT customer_id FROM loan;

-- Customers holding a deposit but no loan
SELECT customer_id FROM fixed_deposit
EXCEPT
SELECT customer_id FROM loan;
```

Operator	Returns	Oracle's name
INTERSECT	Rows present in both	INTERSECT
EXCEPT	Rows in the first but not the second	MINUS

EXCEPT and **MINUS** are the same operator. Oracle used **MINUS** long before the standard settled on **EXCEPT**, and Oracle 21c now accepts both. This is a small detail that becomes a real nuisance in Module 11.

4.4 Where set operators earn their keep

```
-- Reconciliation between two systems
SELECT txn_id, account_id, amount FROM core_txn_extract
EXCEPT
SELECT txn_id, account_id, amount FROM warehouse_txn_extract;
```

An empty result means the two systems agree row for row. Any row returned is a discrepancy somebody must explain. Run it in both directions to catch rows present in the warehouse but missing from the core.

This is precisely how you validate a data migration, which is why Module 11 returns to it.

5. The aggregate toolkit

Section 2 covered `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` and the `FILTER` clause. Those five carry most reports. This section is the rest of the toolkit: the functions that turn a group into a list, guard a ratio, or describe a spread.

5.1 Turning a group into a list

`COUNT` tells you how many. `STRING_AGG` tells you which.

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name      AS customer,
       COUNT(*)                               AS accounts,
       STRING_AGG(a.account_number, ', '
                  ORDER BY a.account_number) AS account_list
FROM   customer c
JOIN   account a ON a.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(*) > 1
ORDER BY c.customer_id
LIMIT 2;
```

customer_id	customer	accounts	account_list
1005	Karthik Gupta	2	ICIC0000009005, ICIC0000009006
1010	Priya Verma	2	ICIC0000009011, ICIC0000009012

The `ORDER BY` inside the aggregate is not decoration. Without it the order is whatever the engine happens to produce, which means the same report run twice can differ. On anything a regulator reads, order it.

`ARRAY_AGG` does the same job but returns an array rather than text, which is what you want when a program rather than a person consumes the result.

****Do not use `STRING_AGG` to build a comma separated list you then split apart again.**** That is a repeating group, which Module 4 will teach you to remove. It is for display, not for storage.

5.2 Where `COUNT` and `SUM` disagree

Four ICICI Bank branches have no loans at all. Ask for their loan book with an outer join and the two aggregates answer differently:

```
SELECT b.branch_name,
       COUNT(l.loan_id)           AS loans,
       SUM(l.outstanding_amount)  AS outstanding,
       COALESCE(SUM(l.outstanding_amount), 0) AS outstanding_safe
FROM   branch b
LEFT JOIN loan l ON l.branch_id = b.branch_id
GROUP BY b.branch_id, b.branch_name
ORDER BY b.branch_id
LIMIT 2;
```

branch_name	loans	outstanding	outstanding_safe
Fort Mumbai Main	3	2655077.80	2655077.80
Andheri East	0	[NULL]	0.00

COUNT of a column returns 0 for an empty group. SUM returns NULL. Both are correct: there are no loans to count, and there is no total to give. A report that prints an empty cell where it should print a zero is the usual result, and **COALESCE** is the fix.

Use COUNT(1.loan_id), never COUNT(*), on the outer side of a join.

COUNT(*) counts the branch row itself and reports 1 loan for a branch that has none. This is the single most common outer-join reporting bug.

5.3 Ratios and percentages without dividing by zero

```
SELECT channel,
       COUNT(*)                               AS txns,
       ROUND(100.0 * COUNT(*)
             / NULLIF(SUM(COUNT(*)) OVER (), 0), 2) AS pct_of_all
FROM   account_txn
GROUP BY channel
ORDER BY txns DESC
LIMIT 1;
```

channel	txns	pct_of_all
RTGS	71	14.34

Three things are doing work here. **100.0** rather than **100** forces the division to be decimal rather than integer. **NULLIF** turns a zero denominator into NULL so an empty table returns NULL instead of raising **division by zero**. And the **OVER ()** gives the grand total beside each group, which is a window function, the subject of Module 6.

5.4 Describing a spread, not just a middle

An average alone hides everything. The ICICI Bank deposit book is a good example: the average account holds far more than the typical account.

Function	Answers
AVG(x)	The arithmetic mean, dragged by outliers
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)	The median, the true middle
STDDEV(x)	How widely values are spread around the mean
VARIANCE(x)	The square of the standard deviation
MODE() WITHIN GROUP (ORDER BY x)	The most frequent value

```
SELECT COUNT(*)                               AS accounts,
       ROUND(AVG(balance), 2)                 AS mean,
       ROUND(PERCENTILE_CONT(0.5)
```

```

        WITHIN GROUP (ORDER BY balance)::NUMERIC, 2) AS median,
        ROUND(STDDEV(balance), 2) AS std_dev,
        MIN(balance) AS smallest,
        MAX(balance) AS largest
FROM   account;

```

```

accounts | mean      | median    | std_dev  | smallest | largest
-----+-----+-----+-----+-----+-----
      36 | 220372.35 | 215657.32 | 142873.59 | 13314.15 | 473345.84

```

The standard deviation here is almost two thirds of the mean, which tells you the book is very unevenly spread. A single average balance quoted to a product manager would be misleading, and this is why a risk report shows percentiles.

`PERCENTILE_CONT` and `MODE` are **ordered-set aggregates**: they need `WITHIN GROUP (ORDER BY ...)` because a median is meaningless without an order.

5.5 MIN and MAX are not GREATEST and LEAST

This catches almost every batch.

Function	Works down	Example
<code>MIN(x)</code> and <code>MAX(x)</code>	A column , across many rows	The largest balance in the bank
<code>LEAST(a, b)</code> and <code>GREATEST(a, b)</code>	A row , across several columns	The later of two dates on one row

```

-- across rows: one answer for the whole table
SELECT MAX(balance) AS largest_in_bank FROM account;

-- across columns: one answer per row
SELECT loan_id,
       GREATEST(disbursed_date, DATE '2026-01-01') AS effective_from
FROM   loan
LIMIT 3;

```

Using `MAX` where you meant `GREATEST` collapses your report to a single row, which is usually noticed. Using `GREATEST` where you meant `MAX` gives you one row per input row, which often is not.

5.6 Aggregating dates

`MIN` and `MAX` work on dates and timestamps, and that is how you find a customer's first and last activity.

```

SELECT account_id,
       COUNT(*) AS txns,
       MIN(txn_date) AS first_txn,
       MAX(txn_date) AS last_txn,
       MAX(txn_date) - MIN(txn_date) AS active_span
FROM   account_txn
WHERE  account_id = 9003
GROUP BY account_id;

```

```

account_id | txns |      first_txn      |      last_txn
-----+-----+-----+-----
      9003 |   16 | 2026-01-02 21:25:00 | 2026-03-27 20:05:00

```

Bucketing by month uses `DATE_TRUNC` from Module 2 in the `GROUP BY`:

```

SELECT DATE_TRUNC('month', txn_date) AS month,
       COUNT(*)                     AS txns,
       ROUND(SUM(amount), 2)        AS value
FROM   account_txn
GROUP BY DATE_TRUNC('month', txn_date)
ORDER BY month;

```

Group by the expression, not by the alias. `GROUP BY month` fails in PostgreSQL for the reason section 6.1 of Module 2 gave: the `SELECT` list is evaluated after `GROUP BY`, so the alias does not exist yet.

5.7 Quick reference

Function	Returns	Ignores NULL
<code>COUNT(*)</code>	Rows in the group	No
<code>COUNT(col)</code>	Rows where col is not null	Yes
<code>COUNT(DISTINCT col)</code>	Distinct non-null values	Yes
<code>SUM, AVG, MIN, MAX</code>	The obvious	Yes
<code>STRING_AGG(col, sep)</code>	The values joined into text	Yes
<code>ARRAY_AGG(col)</code>	The values as an array	No
<code>STDDEV, VARIANCE</code>	Spread around the mean	Yes
<code>PERCENTILE_CONT(f)</code>	The value at that fraction	Yes
<code>MODE()</code>	The most frequent value	Yes
<code>BOOL_AND, BOOL_OR</code>	Whether all or any are true	Yes
<code>COUNT(*) FILTER (WHERE ...)</code>	A conditional count	No

Every aggregate except `COUNT(*)` and `ARRAY_AGG` ignores NULL. Section 2.2 showed why that matters for `AVG`. It matters just as much for `STRING_AGG`, where a missing nominee silently drops out of the list rather than appearing as a gap.

5.8 The same job in three dialects

Job	PostgreSQL	Oracle	SQL Server
Join a group into text	<code>STRING_AGG</code>	<code>LISTAGG</code>	<code>STRING_AGG</code>
Conditional count	<code>COUNT(*) FILTER (WHERE c)</code>	<code>COUNT(CASE WHEN c THEN 1 END)</code>	<code>COUNT(CASE WHEN c THEN 1 END)</code>
Median	<code>PERCENTILE_CONT(0.5)</code>	<code>MEDIAN</code>	<code>PERCENTILE_CONT(0.5)</code>
Sample standard deviation	<code>STDDEV_SAMP</code>	<code>STDDEV</code>	<code>STDEV</code>
Subtotals	<code>GROUP BY ROLLUP</code>	<code>GROUP BY ROLLUP</code>	<code>GROUP BY ROLLUP</code>

`COUNT(CASE WHEN ... THEN 1 END)` works everywhere including PostgreSQL, so it is the portable spelling. `FILTER` is clearer, and Module 11 will convert it.

6. Assembling the performance pack

6.1 Branch summary

```
SELECT  b.branch_name,
        COUNT(DISTINCT c.customer_id) AS customers,
        COUNT(DISTINCT a.account_id) AS accounts,
        SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END) AS money_in,
        SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END) AS money_out
FROM    branch b
        LEFT JOIN customer c ON c.home_branch_id = b.branch_id
        LEFT JOIN account a ON a.customer_id = c.customer_id
        LEFT JOIN account_txn t ON t.account_id = a.account_id
        AND t.status = 'SUCCESS'

GROUP BY b.branch_name
ORDER BY money_in DESC;
```

branch_name	customers	accounts	money_in	money_out
T Nagar Chennai	4	5	2691431.83	1980064.27
Andheri East	4	5	1926049.03	2259434.39
Banjara Hills	4	5	1384840.12	1465615.02
Koramangala	4	4	1334932.58	1580750.78
Connaught Place	3	3	1010101.11	1651285.02
Fort Mumbai Main	4	5	995678.38	2255754.80
Baner Pune	3	4	669734.29	2012828.91
Whitefield	4	5	347121.91	2029982.76

6.2 The fan-out problem

`COUNT(DISTINCT ...)` is not decoration here. It is the difference between a correct report and a wrong one.

Customer 1010 holds 2 accounts, with 14 and 13 transactions.
After the joins, customer 1010 appears in 27 rows.

```
COUNT(c.customer_id)      -> 21   wrong
COUNT(DISTINCT c.customer_id) -> 1   right
```

A join **fans out**: one customer row becomes many rows once accounts and transactions are attached. Any `COUNT` or `SUM` taken after a fan-out counts the multiplied rows unless you handle it deliberately.

The same trap applies to sums. `SUM(a.balance)` in the query above would add each account's balance once per transaction on that account. When you need both a per-account total and a per-transaction total in one report, aggregate them separately and join the two summaries, rather than trying to do both in one pass.

This is the single most common reporting defect in the industry. The number looks plausible, nobody can see the multiplication, and it is only caught when somebody adds it up by hand.

6.3 Dormant customers

```

SELECT  b.branch_name, c.customer_id,
        c.first_name || ' ' || c.last_name AS customer_name, c.phone
FROM    customer c
        JOIN branch b ON b.branch_id = c.home_branch_id
WHERE   NOT EXISTS (
        SELECT 1
        FROM    account_txn t JOIN account a ON a.account_id = t.account_id
        WHERE   a.customer_id = c.customer_id
                AND t.txn_date >= DATE '2026-01-01')
ORDER BY b.branch_name, c.customer_id;

```

`NOT EXISTS` rather than `NOT IN`, because the subquery is over a joined set that could produce NULLs.

Terms introduced in this module

Term	Meaning
Join predicate	The condition in the <code>ON</code> clause
Cartesian product	Every combination of rows from two tables
Anti-join	Outer join plus a NULL test, to find non-matches
Self join	A table joined to itself, for hierarchies
Non-equi join	A join on a condition other than equality
Fan-out	One parent row becoming many after a join
Conditional aggregation	<code>SUM(CASE WHEN ...)</code> , pivoting rows into columns
Correlated subquery	A subquery referencing the outer query, evaluated per row
Derived table	A subquery in the <code>FROM</code> clause
Set operator	<code>UNION</code> , <code>INTERSECT</code> , <code>EXCEPT</code>

Where this module leads

You learned	It becomes
Joins reassemble separated data	Module 4, which explains why it was separated
Correlated subqueries are slow at scale	Module 5, rewriting and indexing
Derived tables	Module 6, common table expressions and window functions
<code>EXCEPT</code> for reconciliation	Module 11, validating a migration
The branch performance pack	Module 10, scheduled as a SQL Server Agent job